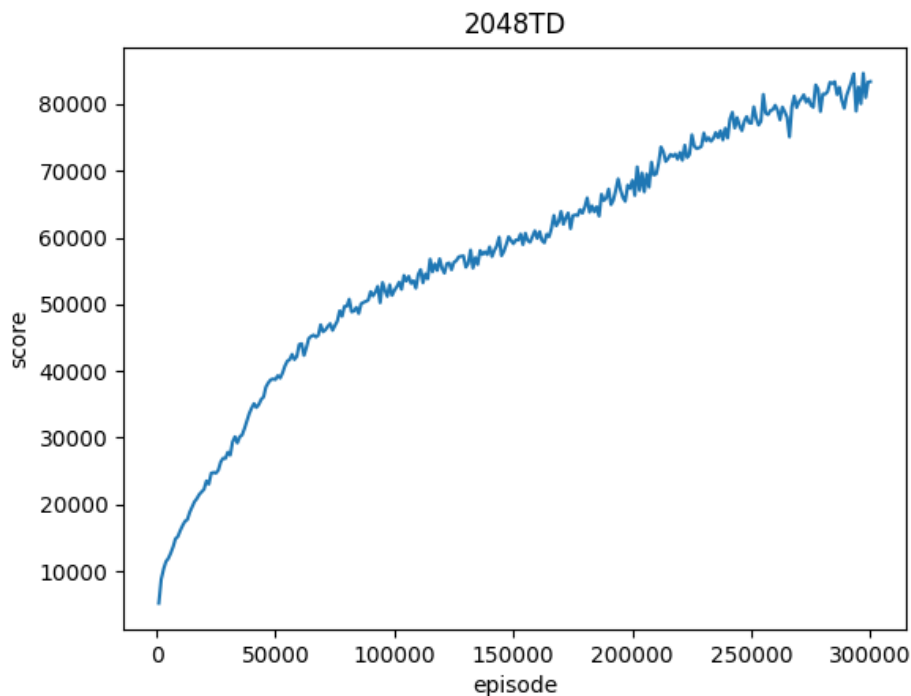


Lab2 : 2048TD

311552055 蕭育泓

1. Training episodes



2. Describe the implementation and n -tuple network

因為 2048 有 16 格，每一格可能會是 0、2、4...，假設他到 2048 總共有 12 種，這樣我們盤面就要有 12^{16} 種組合，所以假設我們把 16 個格子切成 6-tuple 的形式，我們每次都只需紀錄這 6 個格子就好，大小就會變成 12^6 種組合，但是由於我們把圖形做分割，所以就會有同構的問題，所以我們必須將旋轉後的 tuple，也一併納入，但這空間的使用也是遠遠足夠我們來使用的。

```
protected:
    static float* alloc(size_t num) {
        static size_t total = 0;
        static size_t limit = (1 << 30) / sizeof(float); // 1G memory
        try {
            total += num;
            if (total > limit) throw std::bad_alloc();
            return new float[num]();
        } catch (std::bad_alloc&) {
            error << "memory limit exceeded" << std::endl;
            std::exit(-1);
        }
        return nullptr;
    }
    size_t length;
    float* weight;
};
```

上圖為，class feature 中，計算我們的 weight table 大小。

```
for (int i = 0; i < 8; i++) {
    board idx = 0xfedcba9876543210ull;
    if (i >= 4) idx.mirror();
    idx.rotate(i);
    for (int t : p) {
        isomorphic[i].push_back(idx.at(t));
    }
}
```

上突圍在 class pattern 中，考慮同構問題下，紀錄各個 table 中的值。

總歸來說，n-tuple network 就是將我們原本要使用的 tuple 切成較小的 tuple，然後在來實作，這能帶給我們空間上的節省，只是會需要多計算同構問題。

3. Explain the mechanism of TD(0)

就是說我們只考慮下一個盤面的期望值，而非 Monte-Carlo 的 TD-1 完全考慮實際多少分數的結果，因為在 2048 中，我們很容易在同一個盤面最終有很多不同的分數，這會導致我們的 Variance 很大，所以使用 TD(0)會相對穩定。

$$\widetilde{V}(s) \leftarrow V(s) + \alpha(r + V(s') - V(s))$$

上圖為 TD(0) 的 algo，實作時，我們會從倒數的第二個 state 開始，將下個的 state 的 value 減去目前這個 state 的 value，計算出他的 error 後，再利用 reward 與 learning rate 不斷更新每個 state，直到每個 state 收斂。

4. Describe your implementation in detail

這次主要有 5 個 TODO 要去做，分別是以下幾點：

1.

```
size_t indexof(const std::vector<int>& patt, const board& b) const {  
    // TODO  
    size_t index = 0;  
    for (size_t i = 0; i < patt.size(); i++)  
        index |= b.at(patt[i]) << (4 * i);  
    return index;  
}
```

這個 Function 主要是去拿到我們 weight table 的 index 值，

而且我們有 16 個格子，所以用 4 bits 來存，在使用 6-tuple 下，總共就會有 24bits。

2.

```
virtual float estimate(const board& b) const {  
    // TODO  
    float value = 0;  
    for (int i = 0; i < iso_last; i++) {  
        size_t index = indexof(isomorphic[i], b);  
  
        value += operator[](index);  
    }  
    return value;  
}
```

因為我們有 8 種同構情形，所以 `iso_last = 8`，所以我們利用剛剛 1. 的 function 去拿 8 個同構的 index，再利用 operator 拿到所要 board 的 weight，最終加總後得到這個當前的 value。

3.

```
virtual float update(const board& b, float u) {  
    // TODO  
    float adjust = u / iso_last;  
    float value = 0;  
    for (int i = 0; i < iso_last; i++) {  
        size_t index = indexof(isomorphic[i], b);  
        operator[](index) += adjust;  
        value += operator[](index);  
    }  
    return value;  
}
```

這裡在做的就是更新每個 pattern，因為我們有 8 個同構

pattern，所以會先除以 8，然後再把這個值(adjust)，利用剛剛 2.

的方法，反向加回給每個 pattern，最終再回傳每個更新過

pattern 的 weight 加總。

4.

```
// TODO
std::vector<int> space;
board table = move->after_state();
for(int i = 0; i < 16; i++){
    if(table.at(i) == 0){
        space.push_back(i);
    }
}

int num_space = space.size();
float expect = 0;
for(int i=0;i<num_space;i++){
    board buf1 = table;
    board buf2 = table;
    buf1.set(space[i],1);
    expect += (0.9 * estimate(buf1));
    buf2.set(space[i],2);
    expect += (0.1 * estimate(buf2));
}
expect /= num_space;
// std::cout << expect << std::endl;
move->set_value(move->reward() + expect);
// move->set_value(expect);
if (move->value() > best->value())
    best = move;
```

這邊在做的是 select the best move，所以我們要先去計算出這個 action 過後，我們的 after state 中，應該會出現哪些空格，這些空格有可能會產生 2 或 4 的新格子，所以我們要去計算出他的這個 action 過後的期望值，最終由下圖公式得知，把 move 的 value 設為 reward + 期望值。

Making a decision (based on value).

$$\pi(s) = \operatorname{argmax}_a (R_{t+1} + \mathbb{E}[V(S_{t+1}) \mid S_t = s, A_t = a])$$

5.

```
void update_episode(std::vector<state>& path, float alpha = 0.1) const {  
    // TODO  
    float target = 0;  
    for(path.pop_back();path.size();path.pop_back()){  
        state& move = path.back();  
        // TD = r + V(s'[t+1]) - V(s'[t])  
        float TD_error = move.reward() + target - estimate(move.before_state());  
        target = update(move.before_state(), alpha * TD_error);  
    }  
}
```

這裡就是利用 TD error 由後往前來更新我們的每個 state 的期望值，公式為下圖。

- Update value $V(S_t)$ toward TD target $R_{t+1} + \gamma V(S_{t+1})$
 $V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$
 - TD error: $R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

而我們每次也都只看當前與下階段的 state value，也就是 TD learning，TD(0)。